

Introduction to Databases

Sistemas de Informação 1

Content

- Examples of use
 - Before databases: Manual System
 - Before databases: File systems
 - Example: Real Estate Agency
 - File system limitations
 - Database
 - DBMS: Database Management System
 - Views
 - Components of a DBMS
 - Roles
 - Advantages and disadvantages of a DBMS
-

Examples of use

- Grocery shopping: scanning barcodes to access prices and check and update stock.
- Credit card purchases: verifying past purchases, card expiration date, and whether the credit limit has been reached.
- Vacation booking: flights and hotels, ensuring that no two different reservations are made at the same time.
- Library usage: users, books, reservations.
- Online shopping: users, products, purchases.
- Studying: students, teachers, subjects, assessments.

Before databases: Manual System

- Forms filled out on paper and organized in some way (e.g., alphabetically).



Before databases: File systems

- Using multiple files

DreamHome
Property for Rent Details
Property Number: PG21

Address <u>18 Dale Rd</u>	Allocated to Branch: <u>163 Main St, Glasgow</u>
City <u>Glasgow</u>	Branch No <u>B003</u>
Postcode <u>G12</u>	Staff Responsible <u>Ann Beech</u>
Type <u>House</u> Rent <u>600</u>	
No of Rooms <u>5</u>	

Owner's Details

Name <u>Carol Farrel</u>	Business Name _____
Address <u>6 Achray St,</u> <u>Glasgow G32 9DX</u>	Address _____
Tel No. <u>0141-357-7419</u>	Tel No. _____
Owner No. <u>C087</u>	Owner No. _____
	Contact Name _____
	Business Type _____

DreamHome
Client Details
Client Number: CR74

First Name <u>Mike</u>	Last Name <u>Ritchie</u>
Address <u>18 Tain St,</u> <u>PA16 1YQ</u>	Tel No. <u>01475-392178</u>

Property Requirement Details

Preferred Property Type <u>House</u>	Maximum Monthly Rent <u>750</u>
General Comments <u>Currently living at home with parents</u> <u>Getting married in August</u>	

Seen By <u>Ann Beech</u>	Date <u>24-Mar-04</u>
Branch No <u>B003</u>	Branch City <u>Glasgow</u>

Example: Real Estate Agency

Rental Department:

- **Property(PropertyNumber, street, city, postalCode, type, bedrooms, rent, OwnerNumber)**
- Owner(OwnerNumber, name, address, phone)
- **Client(ClientNumber, name, address, phone, type)**

Data duplication

Contracts Department

- Rental(RentalNumber, PropertyNumber, ClientNumber, rent, PaymentMethod, StartDate, EndDate)
- **Property(PropertyNumber, street, city, postalCode, type, bedrooms, rent, OwnerNumber)**
- **Client(ClientNumber, name, address, phone, type)**

File system limitations

- Separate and isolated data.
- Duplicate data – wasted space, loss of integrity.
- Data dependency – difficult to change structure.
- Format incompatibility – if different departments use different programming languages to generate files.
- Fixed queries and reports.

Database

A collection of logically related data, and a description of that data, designed to meet the organization's needs.

- Single (possibly large) data repository.
- It can be used simultaneously by different departments and users.
- Instead of separate files with redundant data, all data is integrated, with a minimum number of duplicates.
- The database belongs to the entire organization, rather than just one department.
- It also contains a description of the data: data catalog/dictionary.
- Data abstraction: We can change the definition of the object without affecting the user.
- Composed of entities, attributes, and relationships explicitly shown in an ER diagram.

DBMS: Database Management System

SGBD: Sistema de Gestão de Base de Dados

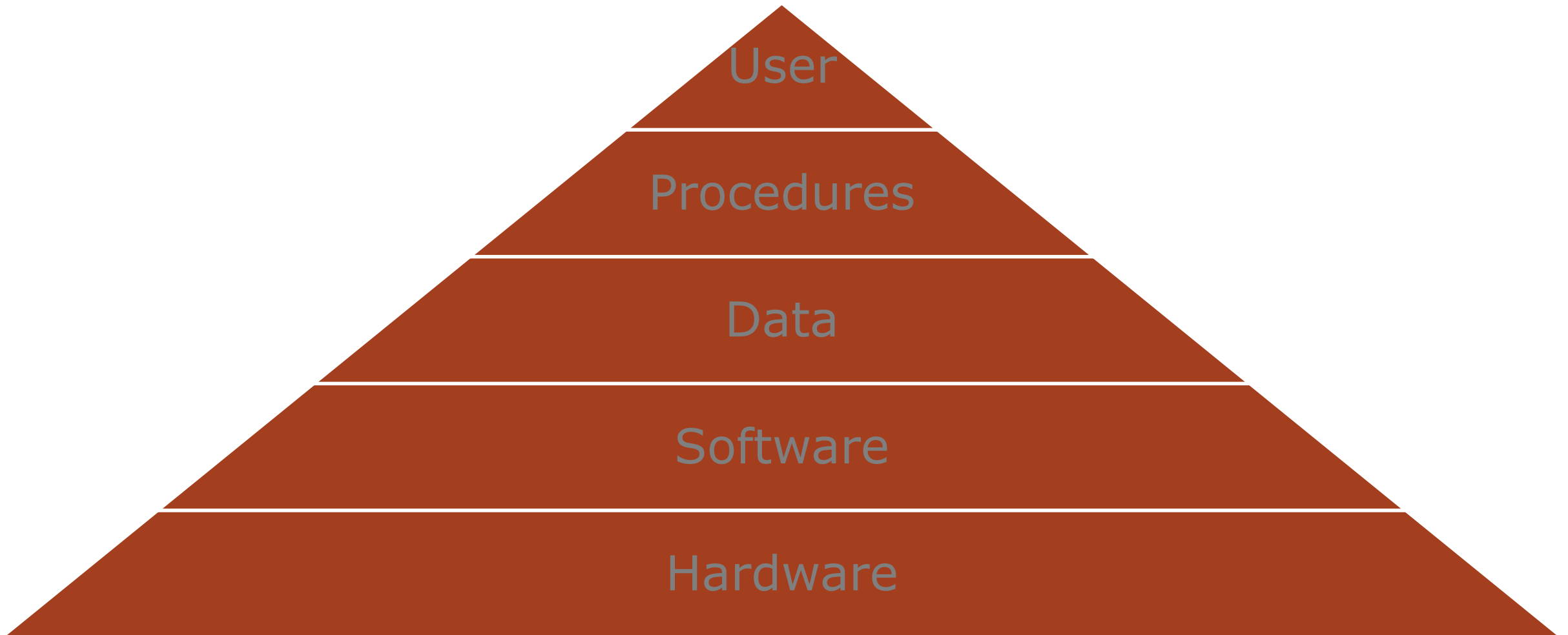
A software system that allows users to define, create, maintain, and control access to the database.

- It allows the definition of the database using a Data Definition Language (DDL), which allows the specification of data types, structures, and constraints.
- It allows inserting, updating, removing, and retrieving data using a Data Manipulation Language (DML). Interaction with the database is done through a query language, e.g., Structured Query Language (SQL).
- It allows controlled access to the database, ensuring:
 - **Security:** only authorized access is allowed.
 - **Integrity:** ensuring that data consistency is maintained.
 - **Concurrency control:** allows simultaneous concurrent access to the database.

Views

- Allow you to reduce data complexity.
- Increase security: not all users should have access to all data.
- Allow you to customize the appearance of the database.
- Present a consistent and unaltered view of the database structure.

Components of a DBMS



Roles

- **Administrator**
 - **DA: Data Administrator**
 - Planning,
 - Development,
 - Maintenance of standards, policies and procedures,
 - Logical/conceptual design.
 - **DBA: Database Administrator**
 - Physical design and implementation,
 - Security and integrity control,
 - Operating system maintenance,
 - Ensuring performance
- **Designers**
 - Identify data, relationships between data, and data constraints.
- **Developers**
 - Create programs that interact with the database.
- **End users**

Advantages and disadvantages of a DBMS

■ Advantages

- Redundancy control
- Consistency
- More information from the same amount of data
- Information sharing
- Greater integrity
- Greater security
- Enforce standards
- Economies of scale (lower cost)
- Balance of conflicting requirements
- Improved accessibility
- Increased productivity
- Easier maintenance
- Improved access concurrency
- Improved backup and recovery systems

■ Disadvantages

- Complexity
- Size
- DBMS cost
- Hardware cost
- Conversion cost
- Performance
- Greatest impact of a failure

Methodology in Database Design

Content

- Database modeling
 - Success factors in database modeling
 - Conceptual Modeling
 - Logical Modeling
 - Physical Modeling
-

Database modeling

■ Conceptual Model

- Build the conceptual representation of the database, including the identification of entities, relationships, and attributes
- Independent of all physical considerations
 - DBMS, applications, programming languages, hardware, performance issues, ...

■ Logical Model

- “Translate” the conceptual model into a logical representation of the database structure, including the design of relationships
- Using a specific data model
- Independent of the DBMS

■ Physical Model

- Decide how the logical structure will be physically implemented in the DBMS
- Database implementation in a specific DBMS

Success factors in database modeling

- Working interactively with users
- Following a structured methodology during the data modeling process
- Using a data-driven approach
- Incorporating structural and integrity considerations into models
- Combining conceptualization, normalization, and transaction validation techniques into modeling methodologies
- Using diagrams to represent as much information as possible
- Using other forms to represent additional information that cannot be represented in diagrams
- Building a data dictionary to supplement the information
- Being willing to repeat steps and processes to meet user requirements

Conceptual Modeling

Step 1: Build the Conceptual Model

- 1.1 Identify Entities
- 1.2 Identify Relationships
- 1.3 Identify and Associate Attributes with Entities and Relationships
- 1.4 Determine Attribute Domains
- 1.5 Determine Candidate, Primary, and Alternate Keys
- 1.6 Consider Using Extended Modeling Techniques (optional)
- 1.7 Check for Redundancy
- 1.8 Validate the Model Based on User Transactions
- 1.9 Review the Conceptual Model with the User

Logical Modeling

Step 2: Build and validate the logical data model

- 2.1 Obtain relationships for the logical model
- 2.2 Validate the relationships with normalization
- 2.3 Validate the relationships based on user transactions
- 2.4 Verify integrity constraints
- 2.5 Reviewing the logical model with the user
- 2.6 Integrate the local logical model with the global model (optional)
- 2.7 Verify the possibility of future growth

Physical Modeling

Step 3: “Translate” the logical model taking the DBMS into consideration

- 3.1 Draw the basic relationships
- 3.2 Design the data representation
- 3.3 Design general constraints

Step 4: Designing file organization and indexes

- 4.1 Analyze transactions
- 4.2 Choose file organization
- 4.3 Choose indexes
- 4.4 Estimate disk space requirements

Step 5: Design views

Step 6: Design security mechanisms

Step 7: Consider introducing redundancy

Step 8: Monitor and refine the operating system

ER (Entity-Relationship) Diagram

EER (Extended Entity-Relationship) Diagram

Content

- ER Diagram
 - Example
 - Diagram Components: Entities
 - Diagram Components: Relationships
 - Degree of relationships
 - Recursive relationship
 - Diagram components: Attributes
 - Simple and compound attributes
 - Single-valued and multi-valued attributes
 - Derived attributes
 - Key attributes
 - Strong and weak entities
 - Attributes in relationships
 - Constraints
 - EER: Specialization and Generalization
 - EER: Aggregation
 - EER: Composition
-

ER Diagram

- Objective:
 - To ensure we have a precise understanding of the nature of the data and how it is used.
- Non-technical and unambiguous model.
- Identify important data (**entities**) and **relationships** between the data.
- More information about entities and relationships can be added – **attributes**.
- **Constraints** can also be added to entities, relationships, and attributes.
- Modeled using UML.

Example

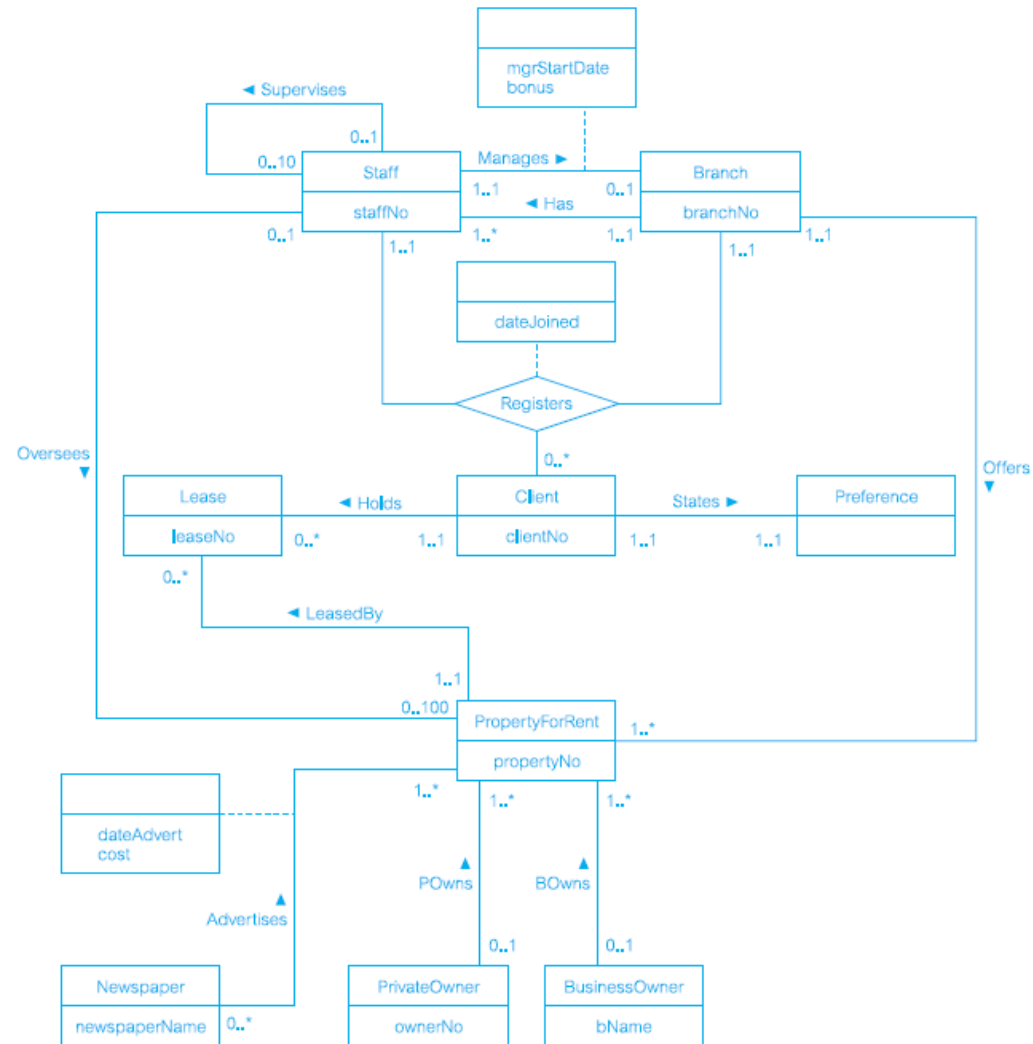


Diagram Components: Entities

Entity: a group of objects with the same properties.
Identified by a name and a list of properties.

Representation:

- Inside a rectangle with the entity's name.
- First letter capitalized.

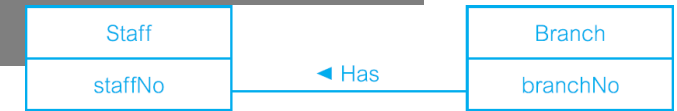
Schema entities:

- Staff
- Branch
- Lease
- Client
- Preference
- PropertyForRent
- Newspaper
- PrivateOwner
- BusinessOwner

Staff

staffNo

Diagram Components: Relationships



Relationship: Association between entities

Each relationship has a name that describes its function.

Names should be unique whenever possible.

The name corresponds to only one of the directions. It is customary to place an arrow to indicate which one.

Degree of relationships: Number of entities that participate in a relationship

Representation:

- Line connecting entities related to an associated verb
- First letter capitalized

Schema relationships

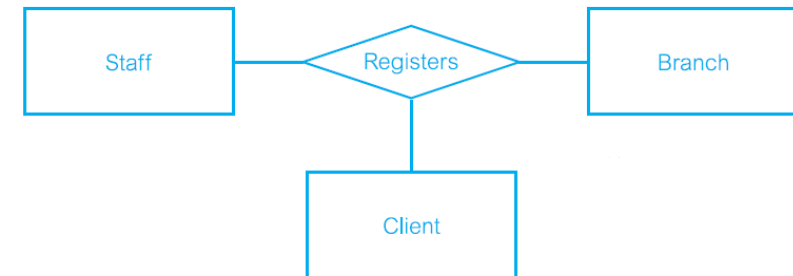
- Supervises
- Manages
- Has
- Oversees
- Offers
- Holds
- States
- LeasedBy
- Advertises
- Powns
- Bowns
- Registers

Degree of relationships

Binary: It has two participating entities.



- Ternary: It has three participating entities.

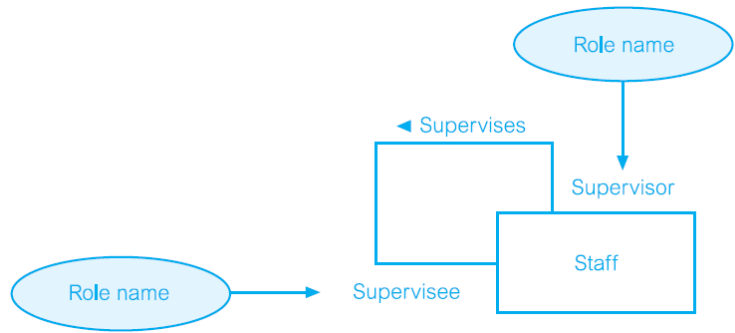


Recursive relationship

A relationship in which the same entity participates more than once.

Also called a unary relationship.

Participating entities can be named to define the role each entity plays in the relationship.



Supervisor: an employee is the supervisor of others
Supervisee: the other employees that are supervised.

Diagram components: Attributes

Properties of entities or relationships

They store the values that represent each instance of an entity.

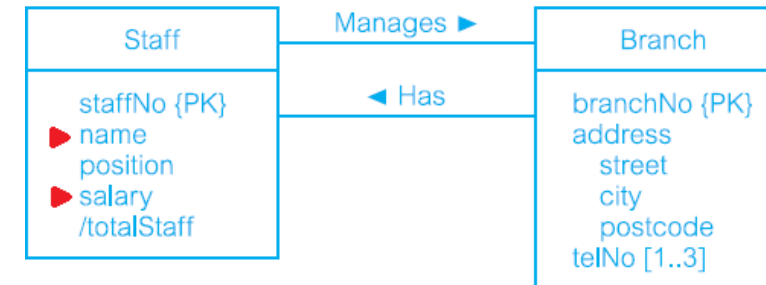
They represent most of the data saved in the database.

Domain of an attribute: the group of values that an attribute can have.

Simple and compound attributes

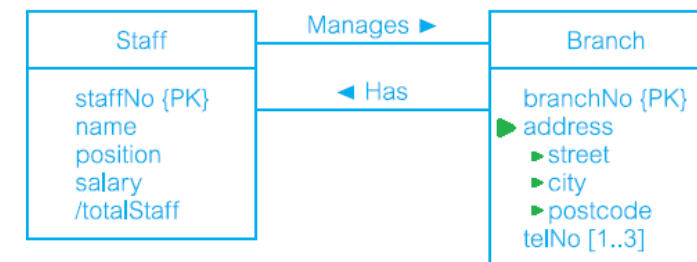
Simple attribute: an independent attribute with only one component.

Example: **name** and **salary** in the Staff entity.



Composite attribute: an attribute composed of multiple components.

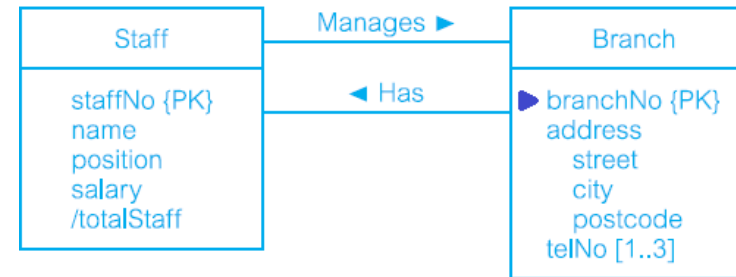
Example: the **address** attribute can be divided into several: **street**, **city**, and **postCode**.



Single-valued and multi-valued attributes

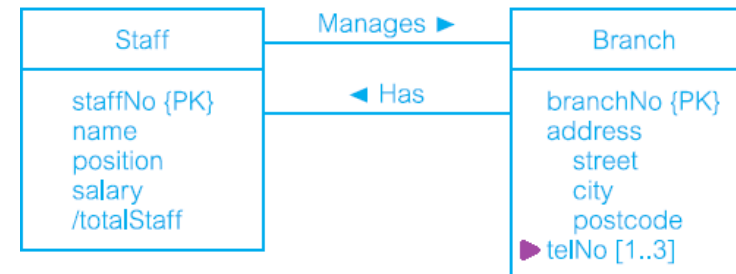
Single-valued attribute: an attribute that has only one value for each occurrence.

Example: the **branchNo** attribute in the Branch entity.



Multi-valued attribute: an attribute that can have multiple values for the same occurrence.

Example: the **telNo** attribute in the Branch entity.

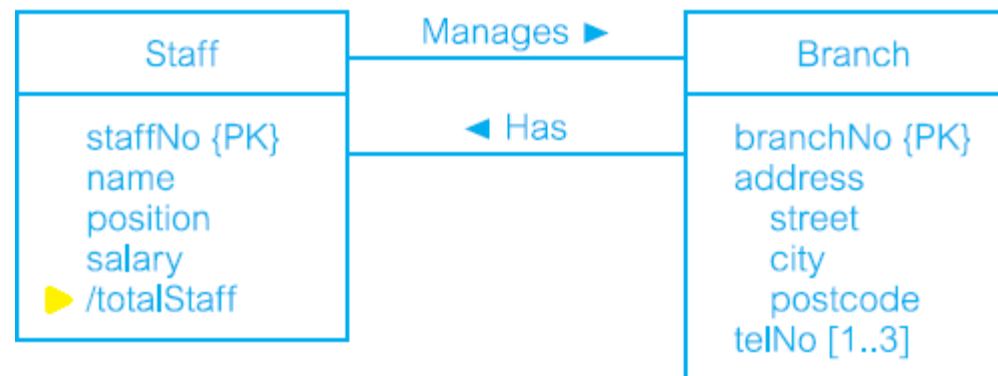


Derived attributes

Attributes that represent a value derived from the values of other attributes, which may or may not belong to the same entity.

Example 1: the value of the **duration** attribute of the Lease entity is calculated from the **rentStart** and **rentFinish** attributes of the same entity.

Example 2: the **totalStaff** value of the Staff entity is derived from the total number of occurrences of the Staff entity.



Key attributes

Candidate keys: the minimum set of values that allow each occurrence of an entity to be uniquely identified. It must contain unique values.

Therefore, it cannot contain the value NULL.

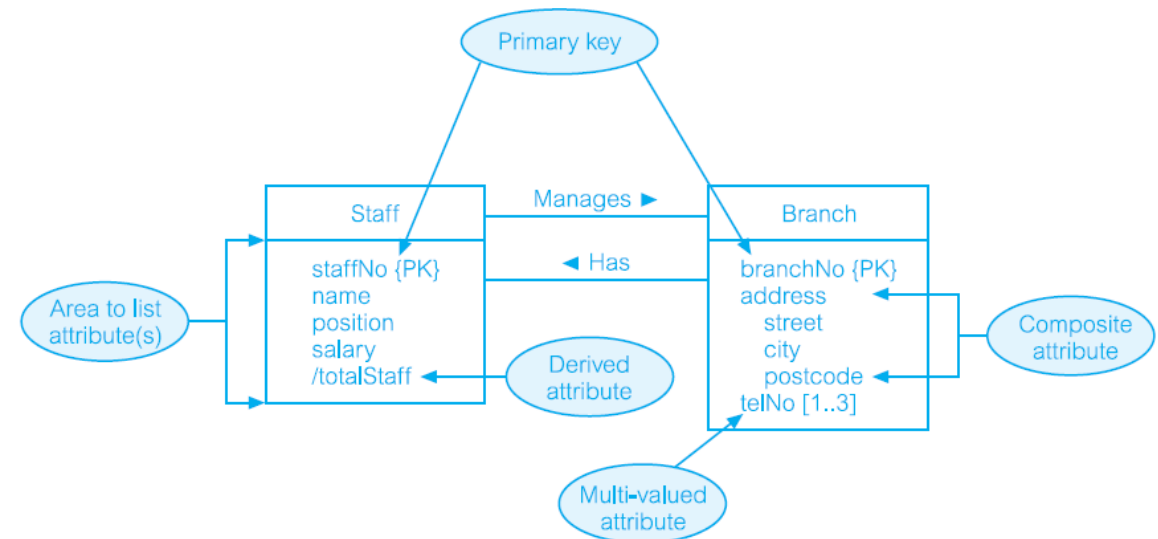
Example: the attribute **branchNo** is a candidate key for the entity Branch.

Primary key: the candidate key that was selected.

Example: an employee can be identified by **staffNo** (e.g., SG14) or by **CPF** (e.g., WL220658D). **staffNo** will be chosen because it is shorter.

Composite key: a key composed of 2 or more values.

Example: the entity Advert with **propertyNo**, **newspaperName**, **dateAdvert**, and **cost**.



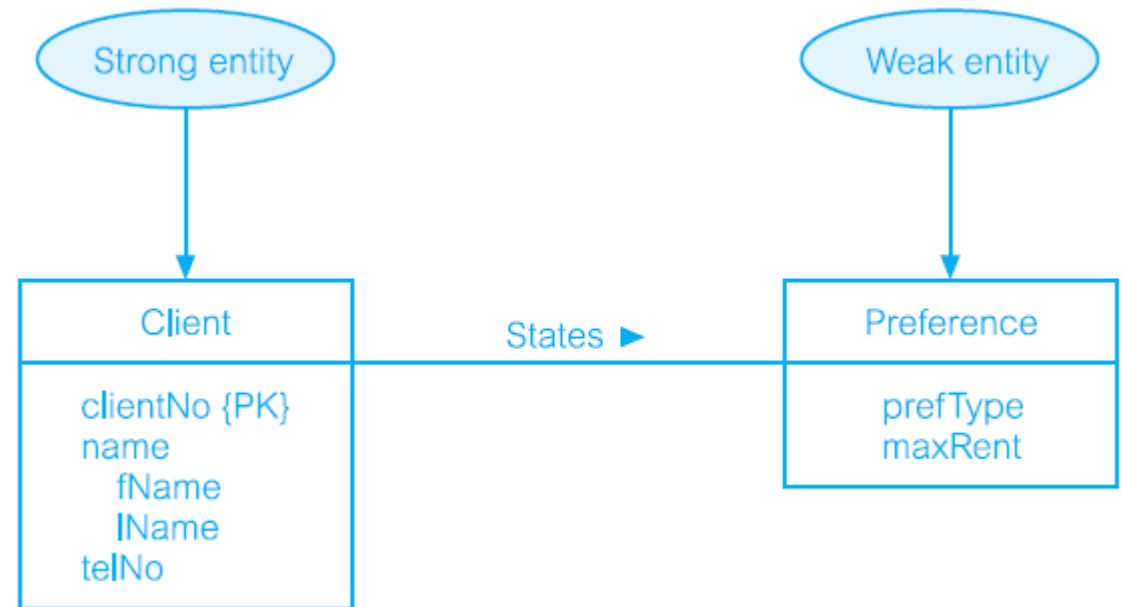
Strong and weak entities

Strong entity: an entity that does not depend on another entity.

Examples: Staff, Branch, PropertyForRent, and Client.

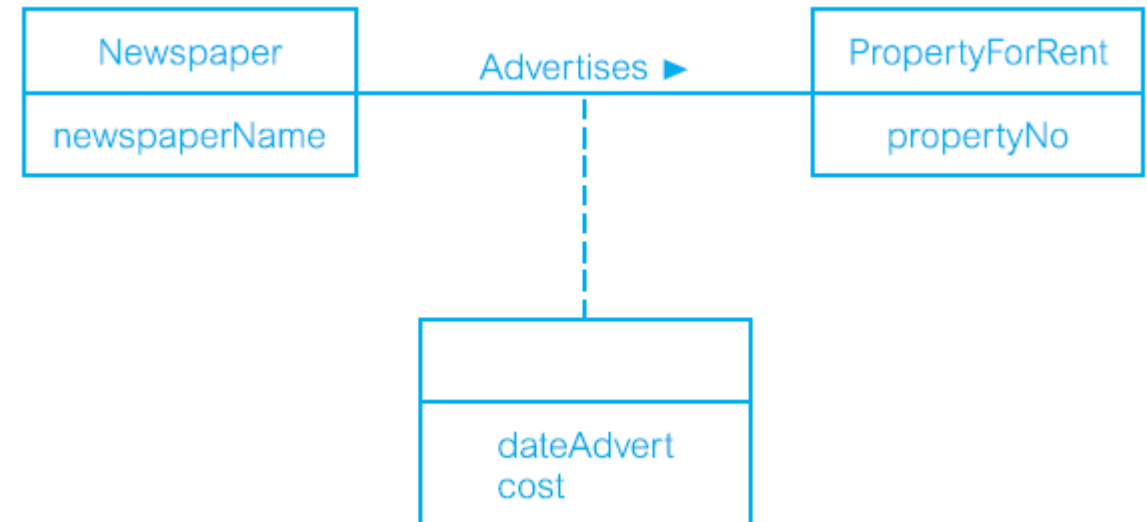
Weak entity: an entity that depends on another entity. Each occurrence of the entity can only be identified when associated with another entity.

Example: Preference.



Attributes in relationships

- Example: the Advertises relationship has attributes dateAdvert and cost.



Representation:

- It uses the same symbol as an entity, but the line linking to the relationship link is dotted.

Constraints

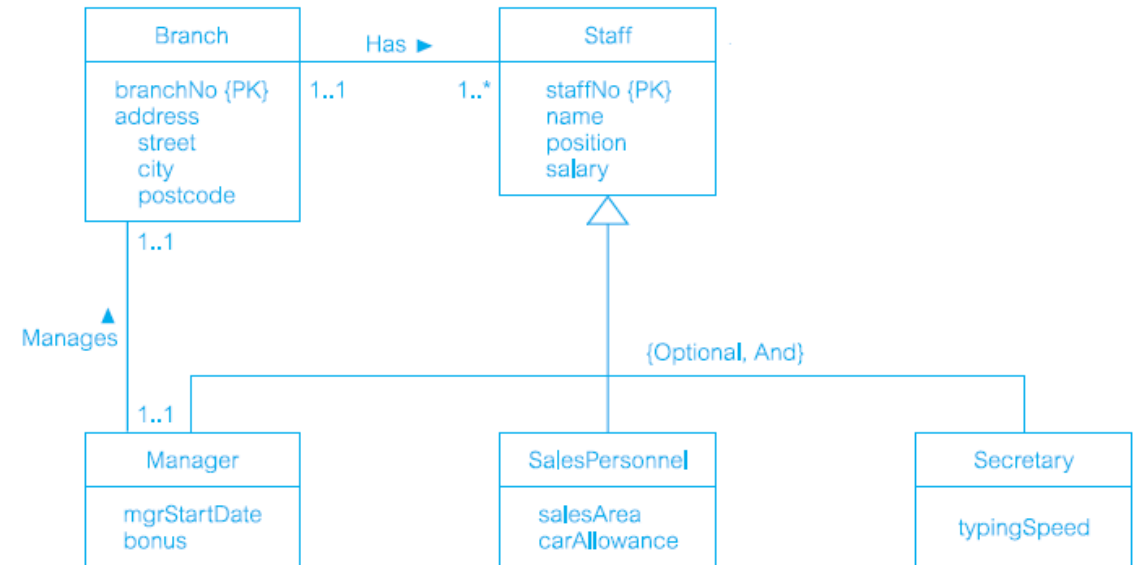
- **Multiplicity:** number (or range) of possible occurrences of an entity in a relationship

Multiplicity	Meaning	Example
0..1	Zero or one	BusinessOwner – 0..1 (Owns) 1..* – PropertyForRent
1..1 (1)	One (exactly)	Client – 1..1 (States) 1..1 – Preference
0..* (*)	Zero or more	Newspaper – 0..* (Advertises) 1..* – PropertyForRent
1..*	One or more	Newspaper – 0..* (Advertises) 1..* – PropertyForRent
n..m	Between n and m	
a,b,c-d	a, b, or vary between c and d.	

- **Cardinality:** maximum number of possible occurrences of an entity in a relationship
- **Participation:** minimum number of possible occurrences of an entity in a relationship
- **Multiplicity in complex relationships** (with more than two entities): number of times one of the entities participates in the relationship, when the multiplicities of the others are fixed.

EER: Specialization and Generalization

- Staff is a generalization of **Manager**, **SalesPersonnel**, and **Secretary**.
- **Manager**, **SalesPersonnel**, and **Secretary** are specializations of Staff.
- Constraints – represented between brackets { }:
 - **Participation**: determines if all members of the superclass must be members of the subclass. Takes values: Mandatory / Optional
 - **Disjoint**: determines if a member of the superclass can be a member of one or more subclasses. Takes values
 - **Or**: if an element of the superclass can only be a member of one subclass
 - **And**: if an element of the superclass can be a member of more than one subclass
- Generalization represented by an arrow on the relationship line



EER: Aggregation

- It represents 'has-a' or 'is-part-of' relationships between entities, where one entity is the whole and the other is the part.

Relationship: Has

Part: Staff

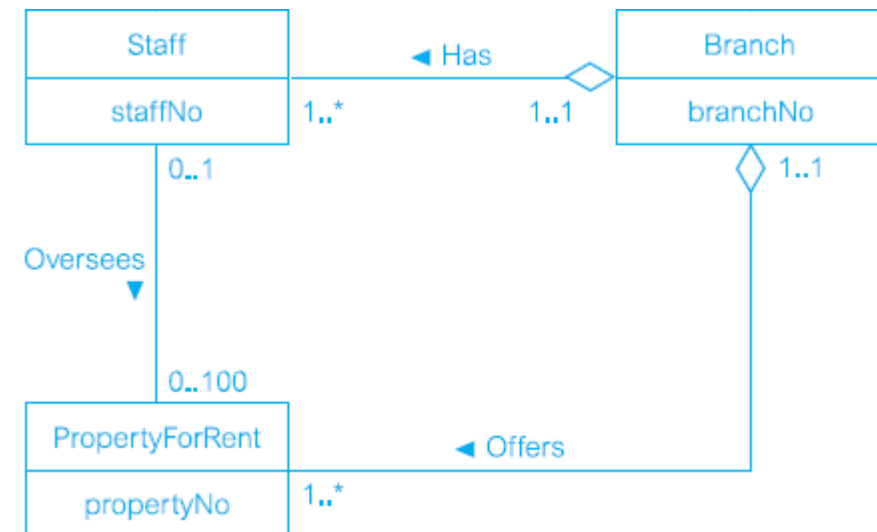
Whole: Branch

Relationship: Offers

Part: PropertyForRent

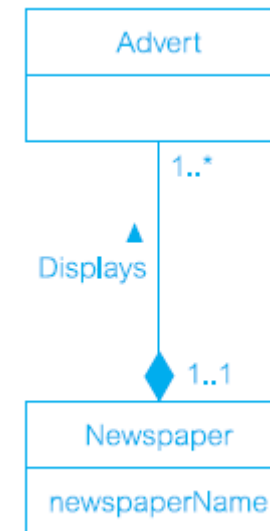
Whole: Branch

- Aggregation represented by a diamond on the line representing the relationship (on the "whole" side)



EER: Composition

- A specific form of aggregation where there is a stronger relationship between the entities.
- Represented as aggregation, but with the diamond filled in.



Relational Model

Content

- Context
 - Terminology
 - Relationships
 - Relationship properties
 - Keys
 - Representation
 - Constraints
-

Context

Relational Model

- Logically structured data in relations (tables).
- Each relation has a name and consists of **attributes** (columns).
- Each **tuple** (row) contains one **value** per attribute.

Objectives:

- Allow for data independence: applications should not be affected by modifications to the internal structure of the data.
- Provide a basis for handling data semantics, consistency, and redundancy.
- Allow for the expansion of data manipulation languages.

att1	att2	att3	att4

Terminology

- **Relationship:** a table with columns and rows (aka: table)
- **Attribute:** a column in a relationship (aka: field)
- **Domain:** the set of allowed values for attributes
- **Tuple:** a row in a relationship (aka: row)
- **Degree** (of a relationship): the number of attributes the relationship contains
- **Cardinality** (of a relationship): the number of tuples the relationship contains
- **Relational database:** a collection of normalized relationships with different relationship names

Relationships

- Relational Table Schema: a named relation defined by a set of attribute/domain pairs. {(attribute: value, ...)}
 - Example – Store: {(branchNo: B005, street: 22 Deer Rd, city: London, postCode: SW1 4EH)}
 - branchNo
 - Street
 - City
 - postCode
- Relational Database Schema: a set of relational table schemas, each with distinct names.

Relationship properties

- Each relation has a different name from all other relations in the relational schema.
- Each cell in a relation contains only one atomic value.
- Each attribute has a different name.
- The values of an attribute are all from the same domain.
- Each tuple is different; there are no duplicate tuples.
- The order of attributes is not meaningful.
- Theoretically, the order of tuples is not meaningful.
 - In practice, the order can affect the efficiency of accessing tuples.

Keys

- **Superkey:** An attribute (or set of attributes) that uniquely identifies a tuple within a relation.
- **Candidate key:** A superkey of minimum size (with the fewest possible attributes).
 - Properties:
 - **Unique:** The key uniquely identifies the tuple.
 - **Irreducible:** No subset of the key can be a key.
 - **Composite key:** A key with more than one attribute.
- **Primary key:** The candidate key that was chosen.
- **Foreign key:** An attribute (or set of attributes) within a relation that corresponds to attributes of another (or the same) relation.

Representation

Convention: RelationshipName(primaryKey, otherPrimaryKey, attribute1, attribute2)

Examples:

- *Branch(branchNo, street, city, postCode)*
- *Staff(staffNo, fName, lName, position, sex, DOB, salary, branchNo)*
- *PropertyForRent(propertyNo, street, city, postCode, type, rooms, rent, ownerNo, staffNo, branchNo)*
- *Client(clientNo, fName, lName, telNo, prefType, maxRent)*
- *PrivateOwner(ownerNo, fName, lName, address, telNo)*
- *Viewing(clientNo, propertyNo, viewDate, comment)*
- *Registration(clientNo, branchNo, staffNo, dateJoined)*

Constraints

- **Null:** represents that the value of an attribute is still unknown or does not apply to the tuple in question.
- **Entity integrity:** the value of a primary key cannot be null.
- **Referential integrity:** if a foreign key exists, one of two things can happen:
 1. The value matches that of the candidate key of another relation.
 2. The value must be null.
 - Example: The Employee table has a foreign key idStore. To add an employee indicating that they work at store L001, that store must already exist in the Store table.